

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	12 July 2026
Project Name	CUSTOMER REGISTRY
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

Reference: <https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/>

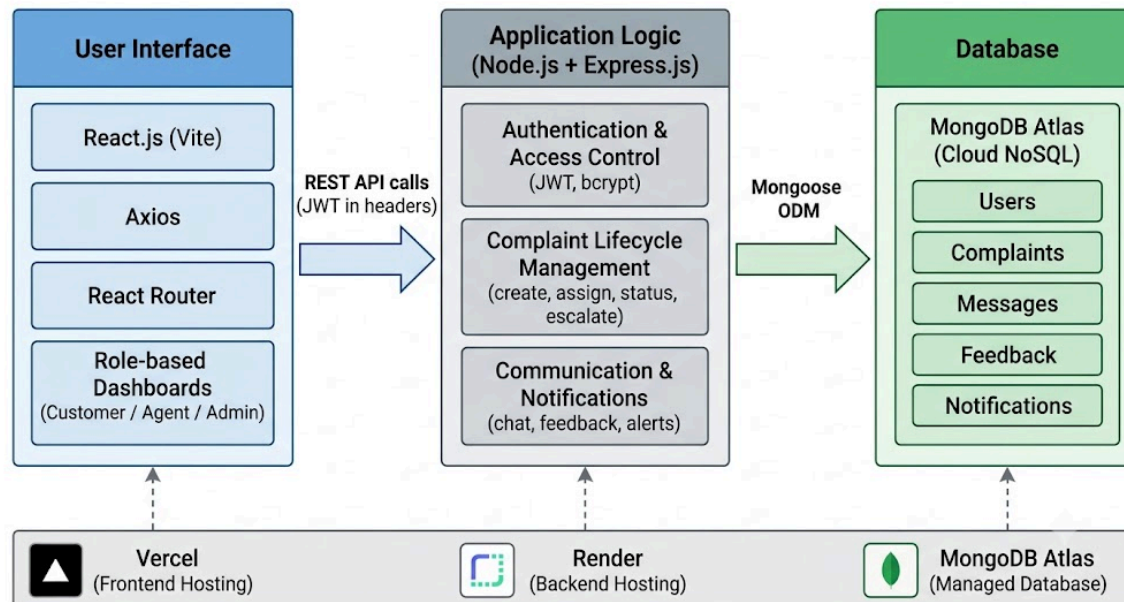


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Web UI for customers, agents, and admins to register, log in, and use role-specific dashboards	React.js (Vite), Axios, React Router, lucide-react
2.	Application Logic-1	Authentication and role-based access control	Node.js, Express.js, JWT, bcrypt.js
3.	Application Logic-2	Complaint lifecycle management — creation, agent assignment, status updates, escalation	Node.js, Express.js, Mongoose (MVC controllers)
4.	Application Logic-3	Chat, feedback, and notification handling	Node.js, Express.js REST endpoints
5.	Database	Storage for users, complaints, messages, feedback, and notifications	MongoDB (NoSQL), Mongoose ODM
6.	Cloud Database	Managed database hosting	MongoDB Atlas
7.	File Storage	Not currently implemented — no file/image upload feature yet	N/A
8.	External API-1	Not currently integrated	N/A — designed for future SMS gateway integration
9.	External API-2	Not currently integrated	N/A — designed for future email service integration
10.	Machine Learning Model	Not applicable to this project	N/A
11.	Infrastructure (Server / Cloud)	Application deployment	Frontend: Vercel · Backend: Render · Database: MongoDB Atlas

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Frameworks and libraries used across the stack	React.js, Express.js, Mongoose, Vite, Axios, lucide-react, bcryptjs, jsonwebtoken, cors
2.	Security Implementations	Authentication, password protection, and access control	JWT-based authentication, bcrypt password hashing, role-based middleware, CORS restrictions
3.	Scalable Architecture	Layered structure separating routes, middleware, controllers, and models	MVC-style REST API (Node.js/Express) + MongoDB
4.	Availability	Frontend via Vercel CDN; backend auto-redeploys from GitHub. Backend on Render free tier (adds latency).	Vercel, Render
5.	Performance	Lightweight JSON REST responses, Vite-optimized frontend bundle	Vite build optimization, indexed MongoDB queries via Mongoose

References:

<https://c4model.com/>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>

